



UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SECP3106-01 APPLICATION DEVELOPMENT (WBL)

SEMESTER 02 2025/2026

LAB ASSIGNMENT 1 REPORT

LECTURER: DR. AHMAD NAJMI BIN AMERHAIDER NUAR

NAME	MATRIC NO
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201

TABLE OF CONTENT

1. .Objective	3
2. Brief description of the system	3
3. Explanation of the interface	3
3.1 The function of the activity input field	3
3.2 The function of the time/category input field	4
3.3 The function of the add button	4
3.4 The function of the activity list	5
3.5 The function of the delete button	6
4. Screenshots of the app	6
5. Conclusion	7

1. .Objective

The purpose of this lab assignment is to apply the basic concepts of Flutter development. Starting with the Flutter basic steps such as creating a project, creating the activity class, and MyApp. Next, creating controllers, list of activities , stateful widgets and set state. Lastly, apply the CRUD feature into the apps. Therefore, by the end of this lab, a simple application is created to enhance understanding in creating the application without database, and complex state management.

2. Brief description of the system

The system's name is daily_activities_recorder_app where it can receive information such as activities, and time/category from the user and display it in the activities list. Then, the user could view the number of listed activities, edit and delete activities that they have created previously. Therefore, this system is a friendly-user system application.

3. Explanation of the interface

3.1 The function of the activity input field

```
class _ActivityPageState extends State<ActivityPage> {  
  // Controllers to manage input  
  final TextEditingController nameController = TextEditingController();  
  
  body: Padding(  
    padding: const EdgeInsets.all(16),  
    child: Column(  
      children: [  
        buildTextField(  
          controller: nameController,  
          label: 'Activity Name',  
          icon: Icons.edit,  
        ),  
      ],  
    ),  
  ),  
}
```

ActivityPageState needs to be stateful because this page is for text input changes, list of activity changes, and deleting activity changes UI. Inside the _ActivityPageStates, there are controllers named nameController that are written to manage the activity text input. It can read what user type, and clear the field. Then inside the widget is how to design its interface.

3.2 The function of the time/category input field

```
final TextEditingController categoryController = TextEditingController();

buildTextField(
  controller: categoryController,
  label: 'Time/Category (e.g., 8:00 PM | Exercise)',
  icon: Icons.category,
),
const SizedBox(height: 8),
```

Next, inside the `_ActivityPageStates`, there are controllers named `categoryController` that are written to manage the category text input. It can read what user type, and clear the field. Then inside the widget is how to design its interface.

3.3 The function of the add button

```
// Add or Update Logic
void addOrUpdateActivity() {
  final String name = nameController.text.trim();
  final String category = categoryController.text.trim();

  if (name.isEmpty || category.isEmpty) {
    showMessage('Please fill in all fields');
    return;
  }
}
```

Trim is used to remove extra spaces at the beginning and end. Then, it validates if any field is empty. If any field is empty, the error message is shown and stops the function by using return.

```
void addActivity(String name, String category) {
  setState(() {
    activities.add(Activity(name: name, category: category));
  });
  clearFields();
  showMessage('Activity added successfully');
}
```

setState () works when the activity list changes, so Flutter rebuilds the UI. Otherwise, only internally change but the screen would not update.

3.4 The function of the activity list

```
Expanded(  
  child: activities.isEmpty  
    ? const Center(child: Text('No activities recorded yet'))  
    : ListView.builder(  
      itemCount: activities.length,  
      itemBuilder: (context, index) {  
        final activity = activities[index];  
        return Card(  
          margin: const EdgeInsets.only(bottom: 12),  
          elevation: 3,  
          child: ListTile(  
            leading: const CircleAvatar(child: Icon(Icons.check)),  
            title: Text(activity.name),  
            subtitle: Text(activity.category),  
            trailing: Row(  
              mainAxisAlignment: MainAxisAlignment.min,  
              children: [  
                IconButton(  
                  icon: const Icon(Icons.edit, color: Colors.blue),  
                  onPressed: () => editActivity(index),  
                ), // IconButton  
                IconButton(  
                  icon: const Icon(Icons.delete, color: Colors.red),  
                  onPressed: () => deleteActivity(index),  
                ), // IconButton  
              ],  
            ), // Row  
          ),  
        );  
      },  
    ),  
)
```

If the list is empty (activities.isEmpty), it will show “No activities recorded yet”. Otherwise, the list is using ListView.builder where it will display listed activities efficiently.

3.5 The function of the delete button

```
void deleteActivity(int index) {  
    setState(() {  
        activities.removeAt(index);  
        // Reset if we were editing the deleted item  
        if (isEditing && editingIndex == index) {  
            cancelEdit();  
        } else if (isEditing && editingIndex > index) {  
            editingIndex--;  
        }  
    });  
    showMessage('Activity deleted');  
}
```

If the user is editing the same activity that gets deleted, edit mode stops and the form is canceled. Otherwise, if editing index = 2, delete index = 1, after deleting index 1, the list would shift. So the edited activities move from index 2 to index 1. That is how editingIndex -- functioned. setState () is used because after deletion, the UI must be updated.

4. Screenshots of the app





5. Conclusion

In conclusion, developing the Daily Activities Recorder application has strengthened the fundamentals of creating a Flutter project, from initial setup to implementing a functional user interface. By applying the concepts from Lesson 1, my understanding of how widget tree and layout structures function has enhanced. Next, applying my lesson 2 knowledge on how to handle user input through controllers and manage data using models and lists and apply how the `setState` and simple widgets works. Lastly, implement the CRUD functions to the application for better apps functionalities.